Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Huffman'da ikili karakterler ve görüntüde Huffman (derste sorulan sorulardan)
  - Huffman ağacında tek karakterlerin yanına metinde en sık geçen birkaç ikili karakter (İngilizcede "th" gibi) de ayrı sembol olarak eklenebilir. Kodlarken önce iki karakter okunur; ikilinin kodu varsa o kullanılır, yoksa tek karakter kodlanır.
  - Eşik çok düşük tutulursa sembol sayısı artar, ağaç büyür ve işlem zorlaşır. Birden fazla ağaç kullanmak algoritmayı çok karmaşıklaştırır.
  - Görüntüde semboller piksel renk değerleridir. True color bir resimde Red, Green ve Blue kanalları ayrı ayrı kodlanmalıdır. 256 renkli (color table kullanan) bir resimde ağaçta 256 sembol ve 256 kod olur.
  - Kayıpsız sıkıştırmada satır başı (carriage return), tab gibi görünmeyen karakterler de sembol olarak alınıp kodlanmalıdır; yoksa açılan metin orijinalle aynı olmaz.
- Önceki yöntemlerin kısa tekrarı
  - Ad hoc yöntemler yalnız uygulamanın ve verinin türüne özgü geliştirilir. İstatistiksel yöntemler sembollerin olasılık ya da frekans değerlerini kullanır.
  - Entropi, kaynağı kodlamak için gereken ortalama bit sayısıdır. Ortalama mesaj uzunluğu, her sembolün bit uzunluğunun olasılığıyla çarpılıp toplanmasıyla bulunur. Redundancy ikisinin farkıdır; ne kadar küçükse yöntem o kadar iyidir.
  - Prefix kod: bir sembolün kodu başka bir sembolün kodunun ön eki olmaz; bu sayede geri dönüşüm mümkündür.
  - Huffman statik, semi-adaptif ve adaptif olabilir. Canonical Huffman, kod uzunluklarını ve her uzunlukta kaç sembol olduğunu kullanarak başlık bilgisini daha az yerle gönderir.
  - String boyu arttıkça sembol başına ortalama kod uzunluğu düşer; buradan sözlük tabanlı yöntemlere geçilir.
- LZ ve türev algoritmalar (ekrandan, 00:20:00)
  - Jacob Ziv ve Abraham Lempel 1970'li yıllarda LZ77 ve LZ78 adını verdikleri ilk sözlük tabanlı sıkıştırma algoritmalarını geliştirdi.
  - Buradaki sözlük, bir dilin anlamlı kelimelerinden oluşan bir sözlük değildir; metinden çıkan, anlamlı olabilen ya da olmayan string'lerdir.
  - Türevler: LZSS (Storer-Szymanski), LZFG (Fiala-Greene), LZRW1 (Ross-Williams), LZW (Welch), LZMW (Miller-Wegman), LZAP (All prefix), LZY (Yabba), LZP (Prediction). Türevler çoğunlukla parametrelerle (buffer boyları, offset ve length alanları, token biçimi) oynar.
- LZ77: kayan pencere (sliding window)
  - Ana fikir: input stream'den önceden okunup kodlanmış bilgiyi sözlük olarak kullanmak.
  - Pencere iki parçadır: solda arama alanı (search buffer, sözlük; en son okunan ve kodlanan semboller), sağda kaynak kod alanı (look-ahead buffer; henüz kodlanmamış semboller). Veri sağdan sola akar; arama alanı dolunca en eski semboller pencereden çıkar.
  - Pratikte arama alanı binlerce byte, kaynak kod alanı onlarca byte olur; arama alanı her zaman daha büyüktür.
  - Token üç alanlıdır: (offset, çakışan sembol sayısı, çakışmadan sonra gelen sembol).
  - Çakışma yoksa offset ve uzunluk 0 yazılır, sembolün kendisi üçüncü alana konur. Başlangıçta arama alanı boş olduğu için ilk token'lar hep böyledir.

  - Kodlayıcı en uzun eşleşmeyi arar. Aynı uzunlukta birden fazla eşleşme varsa türeve göre en yakındaki ya da en uzaktaki seçilir.
- LZ77 örnekleri (ekrandan, 00:30:00 ve 00:46:10)
  - "sir sid eastman easily t | eases sea sick seals": "e" arama alanında sağdan 8. ve 16. konumda bulunur; "eas" üç sembol olarak eşleşir. En uzaktaki çakışma referans alınarak (16, 3, "e") token'ı hazırlanır.
  - "sir sid eastman ..." baştan kodlanınca: (0,0,"s"), (0,0,"i"), (0,0,"r"), (0,0,"␣"), (4,2,"d"), (4,1,"e"), (0,0,"a"), (10,1,"t"), (0,0,"m"), (4,1,"n").
  - Kod çözme aynı token'larla yapılır: offset kadar geri gidilir, uzunluk kadar sembol kopyalanır, arkasına üçüncü alan eklenir.
- Token uzunluğu (ekrandan, 00:40:00)
  - Arama alanı uzunluğu S ise offset $\lceil \log_2 S \rceil$ bit, kaynak kod alanı uzunluğu L ise length alanı $\lceil \log_2 (L-1) \rceil$ bit, sembol alanı 8 bittir.
  - Pratikte offset 10-12 bit, length birkaç bit olur. Yaklaşık token uzunluğu $11+5+8 = 24$ bittir.
- LZ77'nin özellikleri ve türevleri
  - Kod çözücü kodlayıcıdan daha basittir; kodlayıcının pencere boyuna eşit bir buffer yeterlidir. Bu yüzden arşiv uygulamalarına elverişlidir.
  - Birbirine yakın tekrar eden örüntülerde (Türkçedeki sık heceler gibi) performans artar.
  - Türevler offset ve length alanlarıyla, buffer boylarıyla ve pencere yapısıyla oynar.
- Dairesel kuyruk (circular queue) (ekrandan, 01:00:00)
  - Doğrusal pencerede yeni sembol gelince bütün semboller kaydırılır. Dairesel kuyrukta "s" (start) ve "e" (end) olmak üzere iki pointer vardır.
  - Karakterler end pointer'dan eklenir, start pointer'dan çıkarılır. Yeni semboller eklendikçe pointer'lar yer değiştirir; kaydırma yapılmaz.
- LZ78 (ekrandan, 01:20:00)
  - Kayan pencere, arama ve kaynak kod alanı yoktur. Önceden karşılaşılmış string'lerden oluşan, başlangıçta boş bir sözlük vardır; boyu bellekle sınırlıdır.
  - Token iki alanlıdır: (sözlükteki sıra, sembol). Sözlüğün 0. sırası null'dur.
  - Gelen sembol sözlükte yoksa (0, sembol) yazılır. Varsa bir sonraki sembol eklenerek en uzun eşleşme aranır; token'a en uzun eşleşmenin sırası ve arkasından gelen sembol yazılır, bu yeni string sözlüğe eklenir.
  - "sir sid eastman easily ..." için ilk sözlük girdileri: 1 "s", 2 "i", 3 "r", 4 "␣", 5 "si" (1,"i"), 6 "d", 7 "␣e" (4,"e"), 8 "a", 9 "st" (1,"t"), 10 "m", 11 "an" (8,"n"), 12 "␣ea" (7,"a"), 13 "sil" (5,"l"), 14 "y".
  - Kod çözücü token'ları okurken sözlüğü aynı biçimde kendisi kurar.
  - LZ78, LZ77'den daha hızlı çalışır; sıkıştırma performansları birbirine çok yakındır.
- Trie ağacı ve sözlüğün dolması (ekrandan, 01:30:00)
  - Sözlük için en iyi veri yapısı trie ağacıdır. Kök null'dur; her düğüm (sıra, sembol) tutar. "sil" gibi bir string, sözlükte 13 satır taramak yerine üç adımda (s → i → l) bulunur.
  - Sözlük dolduğunda üç seçenek vardır: gelen semboller sözlüğe eklenmeden kodlanır (FROZEN); bütün ağaç silinip boş ağaçla devam edilir; en az kullanılan string (sayaç tutarak) çıkarılıp yerine yenisi eklenir.
- LZW (ekrandan, 01:40:00 ve 01:50:00)
  - Welch'in katkısıyla LZ77, LZ78 ve türevlerinden daha iyi performans veren yöntemdir; sözlük yöntemi denince genelde LZW akla gelir.
  - Sözlüğe codebook, her girdisine code word denir. Codebook başlangıçta alfabeyle doldurulur; boyu bir parametredir (sınırlı ya da donanımın izin verdiği kadar).
  - Kodlama: sözlükte olan en uzun string bulunur, onun sırası output stream'e yazılır, string'e bir sonraki sembol eklenerek sözlüğe konur; kodlama bu son sembolden devam eder.
  - Örnek: alfabe {a, b, c}, input "abbbaacbbabbaacacaabb". Sözlük: 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa, 11 aca, 12 acaa, 13 abb. Output stream: 0 1 4 0 0 2 5 5 7 11 3 1.
  - Codebook sınırlıysa sözlük dolduktan sonra ekleme yapılmaz; yalnız sözlükte var olan en uzun eşleşmenin sırası yazılır.
  - LZW'de sıkıştırma daha hızlı, açma biraz daha ağırdır.
- Görüntü sıkıştırma (ekrandan, 02:17:30)

  - Görüntüde de aynı yöntemler kullanılır: semboller karakter yerine renklerdir. Fark, görüntüde kayıplı sıkıştırmaya izin olmasıdır; gözün algılamadığı renkler atılabilir.
  - Kayıpsız sıkıştırma genellikle pikseller arası korelasyonun düşük olduğu şekil gibi görüntülerde, kayıplı sıkıştırma fotoğraf gibi karmaşık ve yüksek korelasyonlu görüntülerde kullanılır.
  - Kayıpsız görüntü sıkıştırma olasılık ya da sözlük tabanlı ya da ikisini birlikte kullanan bir yöntemle yapılır. PNG, LZ77 ve Huffman'ı birlikte kullanan DEFLATE yöntemini kullanır. JPEG-LS ve JBIG de kayıpsız görüntü için tasarlanmıştır.
- Run-length ve faks (ekrandan, 02:20:00)
  - Run-length metinde iyi sıkıştırmaz ama görüntüde, özellikle siyah-beyaz ve gri tonlu resimlerde başarılıdır. PCX ve TIFF formatları RLE kullanır.
  - Faks makinelerindeki CCITT T4 (Group 3) standardı RLE ve Huffman'ı bir arada kullanır. İlk piksel her zaman beyaz kabul edilir; 64'ten büyük tekrar sayıları 64'e bölünür, bölüm ve kalan farklı Huffman tablolarıyla kodlanır. Bu kodlama Modified Huffman (MH) ya da G31D diye bilinir.
  - T4, bir önceki satırı dikkate alan iki boyutlu kodlamayı da kullanabilir (G32D). Faksta daha sonra JBIG standardı geliştirilmiştir.
- ZIP, gzip ve diğer arşivleyiciler
  - ZIP ve gzip, LZ77'nin bir türeviyle Huffman kodlamayı birlikte kullanır; kayan sözlük 32 KB, kaynak alanı 258 byte olacak şekilde ayarlanmıştır.
  - PKZIP, ARC gibi arşivleyiciler LZW, Huffman, RLE ve Shannon-Fano'yu farklı birleşimlerde kullanır. Klasördeki metin ve resim dosyaları için ayrı algoritma kullanılmaz.
- JPEG-LS tahmini (ekrandan, 02:27:50)
  - Her piksel için batı (W), kuzey (N), kuzeybatı (NW) komşularına bakılarak bir G tahmini yapılır: $\max(N, W) \leq NW$ ise $\min(W, N)$; slayttaki ikinci koşulda $\max(W, N)$; diğer durumlarda $W + N - NW$.
  - İkinci geçişte (NW, W), (NW, N), (N, NE) piksel çiftlerinin farkları 9 gruba ayrılır ($9^3 = 729$ durum). Her durum kendi düzeltme değerini saklar; sonunda tahmin ile gerçek değer arasındaki fark kodlanır.
- JPEG (ekrandan, 02:30:00)
  - DCT (Ayrık Kosinüs Dönüşümü) tabanlı bir standarttır. Kayıp oranı kalite faktörü (Q faktörü) ile ayarlanır. 24 bit renk derinliğindeki fotoğraflarda çok iyi sonuç verir. Birçok yöntemi bir araya getirdiği için sıkıştırma ve açma süresi tek algoritmalı kayıpsız yöntemlerden uzundur.
  - Adımlar: renk dönüştürme (RGB → YUV), örneklemeyi azaltma, 8×8 bloklara DCT, niceleme (quantization), zigzag tarama, RLE ve Huffman.
- Renk dönüştürme ve örneklemeyi azaltma (ekrandan, 02:35:30)
  - RGB → YUV dönüşümü JPEG için gerekli değildir, ama R, G, B arasındaki korelasyonu azalttığı için daha yüksek sıkıştırma sağlar. Y parlaklığı (luminance), U ve V renkliliği (chrominance) ifade eder.
  - $Y = 0{,}299R + 0{,}587G + 0{,}114B$; $U = -0{,}169R - 0{,}331G + 0{,}500B$; $V = 0{,}500R - 0{,}419G - 0{,}081B$ (V satırı için bkz. Detaylı Açıklamalar).
  - İnsan gözü parlaklığa daha çok, renkliliğe daha az hassastır. Bu yüzden 2×2 bloklarda 4:4:4 yerine 4:2:2 ya da 4:1:1 kullanılabilir (down-sampling).
- DCT ve niceleme (ekrandan, 02:40:00, 02:47:00 ve 02:50:00)
  - Görüntü 8×8 bloklara bölünür; her bloğa ayrı DCT uygulanır ve 64 katsayı çıkar. İlki DC, diğerleri AC'dir. Düşük frekanslar sol üst köşede, yüksek frekanslar sağ alt köşededir.
  - DCT $(-128, 127)$ aralığında çalıştığı için her renk değerinden 128 çıkarılır (T matrisi). $N = 8$ için $M_{0,j} = 1/\sqrt{N}$, $i > 0$ için $M_{i,j} = \sqrt{2/N} \cos\left[\frac{(2j+1)i\pi}{2N}\right]$.
  - Niceleme: $C_{i,j} = \text{round}(D_{i,j}/Q_{i,j})$. Q tabloları önceden hazırlanmıştır; parlaklık ve renklilik için ayrı tablolar vardır ve kalite faktörüne göre değişir. Kayıp bu yuvarlamada oluşur.
  - C matrisinin sağ alt köşesi sıfırlarla dolar. Zigzag taramadan sonra arka arkaya gelen sıfırlar RLE ile çok iyi sıkışır; ardından Huffman uygulanır.
  - Açma: $R_{i,j} = Q_{i,j} \cdot C_{i,j}$, ters DCT ile piksel değerleri bulunur ve 128 geri eklenir.
- JPEG 2000, GIF, TIFF, PNG (ekrandan, 02:53:30)
  - JPEG 2000, DCT yerine wavelet dönüşümü kullanır; istemci-sunucu görüntü uygula-

malarında ve kaynağı sınırlı kablosuz cihazlarda kullanılır.
  - GIF, CompuServe tarafından geliştirilmiştir (1987), LZW tabanlıdır, en fazla 8 bit (256 renk) ile çalışır, renkli fotoğraflarda yetersiz kalır. 87a ve 89a sürümleri vardır; 89a birden çok resmi ve animasyonu destekler.
  - TIFF'in 5.0 sürümü LZW, 6.0 sürümü JPEG kullanır; 24 bit true color resimlerde başarılıdır.

Hocanın Özellikle Vurguladığı Kısımlar

- Kayıpsız sıkıştırmada her sembol kodlanmalı
  - Görünmeyen karakterler de sembol olarak kodlanmazsa açılan dosya orijinalle aynı olmaz ve sıkıştırma kayıplı hale gelir.
- LZ77'de en uzun eşleşme ve offset seçimi
  - Kodlayıcı hep en uzun eşleşmeyi arar. En uzaktaki çakışmayı almak algoritmayı basitleştirir ama offset büyür ve daha fazla bit ister; en yakındakini almak offseti küçültür ama geriye gitmeyi (bağlı listede olduğu gibi) gerektirir. Bu seçim farklı türevlerin doğmasına neden olmuştur.
- Token'daki üçüncü alanın nedeni
  - Başlangıçta hiç çakışma olmadığı için sembolün kendisi üçüncü alana yazılmazsa algoritma ilerleyemez ve ilk sembol hiçbir zaman geri elde edilemez.
- LZ77 başta sıkıştırmaz, metin ilerledikçe kazandırır
  - Bir karakter 8 bitle yazılabilirken token yaklaşık 24 bittir. Metin uzadıkça sıfır offsetler azalır, aynı kelimeler tekrarlandıkça eşleşmeler uzar ve performans artar.
- Trie ağacının önemi
  - Sözlükte bir string'i satır satır aramak yerine trie ile sembol sayısı kadar adımda bulmak hem LZ78'de hem LZW'de işi çok kolaylaştırır.
- Ticari ürünler yöntemleri birlikte kullanır
  - Kimse "yalnız Huffman ile" ya da "yalnız LZW ile" sıkıştırmaz. Ticari sıkıştırma formatlarında derste görülen algoritmalar arka arkaya kullanılır; görüntüde bunlara gözün algılamadığı bilgiyi atan ön işlemler eklenir.

Kısa Tekrar Notları

- LZ77: kayan pencere = arama alanı (sözlük, büyük) + kaynak kod alanı (look-ahead, küçük).
- LZ77 token: (offset, uzunluk, sonraki sembol). Çakışma yoksa (0, 0, sembol).
- Offset $\lceil \log_2 S \rceil$ bit, length $\lceil \log_2 (L-1) \rceil$ bit, sembol 8 bit; yaklaşık $11 + 5 + 8 = 24$ bit.
- Dairesel kuyruk: start ve end pointer; kaydırma yok.
- LZ78 token: (sözlük sırası, sembol). Sözlük 0 = null. LZ78 daha hızlı, performans LZ77'ye yakın.
- Sözlük dolunca: FROZEN, ağacı sil ve baştan başla, en az kullanılanı çıkar.
- LZW: sözlük alfabeyle başlar; token yalnız sözlük sırasıdır. "abbbaacbbabbaacacaabb" → 0 1 4 0 0 2 5 5 7 11 3 1.
- PNG = DEFLATE (LZ77 + Huffman). GIF ve TIFF 5.0 = LZW. ZIP/gzip = LZ77 türevi + Huffman, 32 KB pencere, 258 byte.
- Faks T4 Group 3: RLE + Huffman (MH, G31D); iki boyutlu sürüm G32D.
- JPEG: RGB → YUV, down-sampling, 8×8 DCT (−128), niceleme ($C = \text{round}(D/Q)$), zigzag, RLE, Huffman.
- DC = sol üst katsayı; yüksek frekanslar sağ alt köşede, niceleme sonrası sıfırlanır.

Detaylı Açıklamalar

- LZ77'nin kayan pencere mantığı
  - Input stream sağdan gelir, kodlanan semboller pencerenin sol tarafına, arama alanına geçer ve orası sözlük olarak kullanılır. Sözlüğün sonsuz kaynağı yoktur; dolunca en önce giren sembol pencereden çıkar. Kaynak kod alanından kaç sembol kodlandıysa input stream'den o kadar yeni sembol pencereye alınır; "kayan pencere" adı buradan gelir.

  - Hoca slayttaki örneği şöyle açıkladı: kaynak kod alanının ilk sembolü “e”, arama alanında sağdan sola aranır. İlk “e”, “easily” kelimesinde, sağdan 8. konumda bulunur; ardından “a” ve “s” de eşleşir, “eas” üç sembol olarak yakalanır, dördüncü sembolde (“e” ile “t”) eşleşme biter. Arama alanının devamında “eastman” kelimesinde, 16. konumda, ikinci bir “eas” vardır. Derste gösterilen LZ77 en uzaktaki (son bulunan) çakışmayı kullandığı için token (16, 3, “e”) olur.
  -
- “sir sid eastman” örneğinin kodlanması (ekrandan, 00:40:00 ve 00:46:10)
  - Arama alanı boşken “s”, “i”, “r” ve boşluk sözlükte bulunmaz; her biri (0, 0, sembol) olarak yazılır.
  - Sonra “s” sözlükte bulunur, arkasından “i” de eşleşir, “d” eşleşmez. “s” sağdan 4. konumdadır: (4, 2, “d”).
  - Boşluk 4. konumda bulunur, arkasından “e” gelmez: (4, 1, “e”). “a” sözlükte yoktur: (0, 0, “a”).
  - “s” iki yerde vardır ama hiçbirinin arkasından “t” gelmez; en uzaktaki 10. konumdur: (10, 1, “t”). “m” yoktur: (0, 0, “m”). “a” 4. konumdadır, arkasından “n” gelmez: (4, 1, “n”).
  - Bu token’lar python ile yazılan bir LZ77 kodlayıcısıyla (en uzaktaki eşleşmeyi seçen) doğrulandı.
  - Kod çözme: (0,0,“s”) → s; (0,0,“i”) → i; (0,0,“r”) → r; (0,0,“␣”) → boşluk; (4,2,“d”) → 4 geri gidilir, “si” kopyalanır, “d” eklenir; (4,1,“e”) → boşluk kopyalanır, “e” eklenir; (10,1,“t”) → “s” kopyalanır, “t” eklenir. Böylece “sir sid east…” sorunsuz geri elde edilir.
- Token boyu ve kazanç
  - Hocanın hesabıyla offset 11, length 5, sembol 8 bit olunca token 24 bittir. İki çakışma ve bir sembol (3 karakter) içeren bir token, karakterleri tek tek 8 bitle yazmakla aynı yeri (24 bit) tutar; daha uzun eşleşmelerde kazanç başlar.
  - Ders sonrası not: Hoca pratikteki arama alanını önce “binlerce byte” diye anlattı, sonra slayttaki “birkaç byte” ifadesini “3 byte, 4 byte” diye okudu. $\lceil \log_2 S \rceil$ = 10-12 bit, S = 1024-4096 byte’a karşılık gelir; yani arama alanı büyük olasılıkla birkaç kilobyte’tır.
  - Ders sonrası not: Hoca arama alanının dolma mantığını bir yerde “ilk giren son çıkar” diye söyledi; anlattığı işleyiş ve kendi sonraki ifadesi “ilk giren ilk çıkar”dır (FIFO).
- Dairesel kuyruk
  - Doğrusal pencerede yeni bir sembol gelince en eski sembol dışarı atılır ve bütün semboller birer kaydırılır. Dairesel kuyrukta dizinin sonu başına bağlanmış gibi düşünülür: pencere doluyken gelen sembol, start pointer’ın gösterdiği en eski sembolün yerine yazılır; end pointer oraya, start pointer bir sonraki en eski sembole geçer. Bir eşleşmeyle birden fazla sembol (örneğin “sid” ve arkasından gelen sembol) gelirse en eski o kadar sembolün yerine yazılır.
  -
- LZ78 kod çözme ve trie
  - Kod çözücü (0,“s”) okuyunca “s” yazar ve sözlüğün 1. sırasına koyar; (1,“i”) okuyunca 1. sıradaki “s”ye “i” ekleyip “si” yazar ve 5. sıraya koyar; (4,“e”) okuyunca 4. sıradaki boşluğa “e” ekler. Sözlüğü kodlayıcıyla aynı sırada kendisi kurar.
  - Kodlayıcı “sil” string’ini sözlükte ararken satırları tek tek tararsa 13 kontrol yapar. Trie ağacında kökten “s”ye, oradan “i”ye, oradan “l”ye inilerek üç adımda 13. sıraya ulaşılır.
  - Bir sembol ilk kez tek başına geldiğinde de sıra alanı ve sembol birlikte yazılır. Sözlük büyükse (hocanın örneğinde 500 bin girdi) sıra alanı çok bit tutar; oysa karakter 8 bitle yazılabilirdi. Bu durum özellikle başlangıçta olur.
  -
- LZW örneği adım adım (ekrandan, 01:40:00 ve 01:50:00)
  - Sözlük 0 a, 1 b, 2 c ile başlar. “a” var, “ab” yok → 0 yazılır, 3 = ab. “b” var, “bb” yok → 1, 4 = bb. “bb” var, “bba” yok → 4, 5 = bba. “a” var, “aa” yok → 0, 6 = aa. “a” var, “ac” yok → 0, 7 = ac. “c” var, “cb” yok → 2, 8 = cb. “bba” var, “bbab” yok → 5, 9 = bbab. “bba” var, “bbaa” yok → 5, 10 = bbaa. “ac” var, “aca” yok → 7, 11 = aca. “aca” var, “acaa” yok → 11, 12 = acaa. “ab” var, “abb” yok → 3, 13 = abb. Son “b” → 1.
  - Output stream: 0 1 4 0 0 2 5 5 7 11 3 1. Sonuç python ile doğrulandı.
  - Kod çözmede alfabe ile başlayan sözlük ve (sınırlıysa) sözlüğün boyu başlık bilgisinde

gönderilir; kod çözücü okunan sıraları string'e çevirirken sözlüğü yeniden kurar.

- Sınırlı codebook
  - Hocanın ikinci örneğinde codebook ab (3), bb (4), bba (5), aa (6) eklenince doldu. Sonrasında sözlüğe ekleme yapılmaz; her adımda sözlükte var olan en uzun eşleşmenin sırası yazılır ve eşleşmeyen sembolden devam edilir: a → 0, c → 2, bba → 5, bba → 5, a → 0, c → 2, a → 0, ... Hoca burada durdu. Python ile tamamlanan çıktı: 0 1 4 0 0 2 5 5 0 2 0 2 6 4.
  -
- JPEG ayrıntıları
  - DCT matrisi: $N = 8$ için M formülüyle kurulur. T matrisi, bloktaki her pikselden 128 çıkarılarak bulunur. Slaytta DCT katsayı matrisi $D = M^T T M$, ters dönüşüm $N = \text{round}(M^T R M) + 128$ diye yazılı.
  - Ders sonrası not: Slayttaki M tanımıyla (satır indisi i frekansı gösterir) ileri ve ters dönüşüm aynı biçimde olamaz. Python ile denendi: ileri dönüşüm $D = MTM^T$ alınırsa ters dönüşüm $M^T D M$ orijinal bloğu verir.
  - Ders sonrası not: Hoca bir yerde DC katsayısının "en yüksek frekans" olduğunu, ders sonunda da "AC frekansları bırakmak, DC frekansları yok etmek" dedi. Slayta göre DC sol üst köşedeki katsayıdır ve en düşük (sıfır) frekansı gösterir; niceleme ile sıfırlanan, sağ alt köşedeki yüksek frekanslı AC katsayılarıdır.
  - Ders sonrası not: Slayttaki V satırında G katsayısı "0,149" gibi okunuyor. Gri bir piksel ($R = G = B$) için renklilik 0 olmalı, yani katsayıların toplamı 0 olmalıdır; bu yüzden doğru değer büyük olasılıkla $-0{,}419$'dur.
  - Ders sonrası not: Hoca örneklemeyi azaltmayı anlatırken Y için de 4'ün 2'ye düştüğünü söyledi. Slayttaki gösterimde (4:4:4 → 4:2:2 ya da 4:1:1) parlaklık (Y) 4 örnekte kalır; azaltılan U ve V'dir.
  - Ders sonrası not: PNG'yi ders sonunda "LZW kullanıyor" diye söyledi; kendi slaytı ve dersin başındaki ifadesi PNG'nin DEFLATE (LZ77 + Huffman) kullandığıdır.
  - Ders sonrası not: Hoca JPEG 2000'in wavelet olarak "Haar" dönüşümünü kullandığını söyledi; JPEG 2000 büyük olasılıkla 5/3 ve 9/7 (CDF) wavelet'lerini kullanır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.